Министерство образования Новосибирской области
ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б. С. Галущака»

ОТЧЁТ
по практической работе № 2
«Организация проекта с системой контроля версий и CI/CD»

Профессиональный модуль: ПМ.02 «Осуществление интеграции программных модулей для компьютерных систем»

МДК 02.02 «Инструментальные средства разработки программного обеспечения»

Специальность: 09.02.07 «Информационные системы и программирование»

Выполнил:	Якушев Антон Игоревич
Обозначение группы:	ПР-23.101
Вариант:	22

Новосибирск
2026

1. Тема и цель работы

Тема практической работы: «Организация проекта с системой контроля версий и CI/CD». Цель — получить практический опыт разработки Java-проекта с использованием Git и GitHub Flow, автоматической сборки, модульного тестирования и публикации артефактов средствами GitHub Actions.

2. Задание

В соответствии с методическими указаниями необходимо создать GitHub-репозиторий, организовать работу через feature-ветки и Pull Request, реализовать консольное приложение Todo на Java, написать тесты JUnit 5, автоматически собирать проект и запускать тесты при изменениях основной ветки, публиковать JAR в GitHub Releases и GitHub Packages. Дополнительное задание со страницы 52 методички также выполнено в проекте.

3. Использованные средства

- Java 21 (LTS) — язык разработки и целевая версия проекта.
- Gradle 8.14.6 — система сборки; версия закреплена в gradle/wrapper/gradle-wrapper.properties.
- JUnit 5 — модульные тесты логики списка задач.
- Git и модель GitHub Flow — история изменений, feature-ветки и слияния.
- GitHub Actions — CI, создание Release, публикация Maven-пакета и ночные проверки.
- Gradle Shadow Plugin — создание исполняемого fat-JAR.

4. Ход выполнения работы

4.1. Структура проекта

Проект называется todo-app. Класс TodoList содержит бизнес-логику списка задач, а TodoApp отвечает за консольный ввод и вывод. Тесты расположены в src/test/java/org/example/TodoListTest.java. Конфигурация сборки находится в build.gradle, settings.gradle и gradle.properties. Файлы автоматизации хранятся в .github/workflows/.

src/main/java/org/example/TodoApp.java
src/main/java/org/example/TodoList.java
src/test/java/org/example/TodoListTest.java
.github/workflows/ci.yml
.github/workflows/publish-package.yml
.github/workflows/nightly.yml

4.2. Реализованный функционал

Приложение представляет собой консольный список задач, который хранится в памяти процесса. Индексация начинается с нуля. Реализованы команды:

- add <task> — добавить задачу; начальные и конечные пробелы удаляются, пустое значение не добавляется.
- list — вывести все задачи и отметить выполненные задачи символом [x].
- remove <index> — удалить элемент по индексу с проверкой границ.
- done <index> — отметить задачу выполненной.
- search <text> — найти задачи по подстроке без учёта регистра.
- clear — очистить список задач.
- help и exit — показать команды и завершить приложение.

4.3. Модульное тестирование

В TodoListTest реализованы проверки добавления и нормализации текста, игнорирования null и пустых значений, корректного и некорректного удаления, защитной копии списка, очистки, отметки выполнения и поиска. Тесты используют JUnit Jupiter 5 и запускаются задачей Gradle test; задача build зависит от shadowJar, поэтому сборка включает тесты и формирование исполняемого артефакта.

4.4. Система контроля версий

Использована модель GitHub Flow: основная ветка main содержит интегрированное состояние проекта, а изменения оформлены в отдельных ветках. В локальной истории подготовлены feature-ветки feat/todo-app-implementation, feat/actions, feat/github-packages, feat/extended-todo и feat/nightly-ci, каждая слита в main отдельным merge-коммитом. После публикации в GitHub эти ветки можно отправить командой git push origin --all.

4.5. CI и GitHub Releases

Workflow .github/workflows/ci.yml запускается при push в main и при Pull Request в main. Он устанавливает JDK 21, запускает ./gradlew clean build и, только при успешной сборке после push в main, создаёт GitHub Release с тегом v<version> и прикрепляет fat-JAR build/libs/todo-app-<version>.jar. При падении тестов последующий шаг публикации не выполняется.

4.6. Публикация в GitHub Packages

Workflow .github/workflows/publish-package.yml запускается после завершения CI / Build & Release. Условие успешного завершения workflow и ветки main предотвращает публикацию непроверенных изменений. Gradle Maven Publish публикует собранный fat-JAR с координатами org.example:todo-app:0.2.0. Для публикации используются GITHUB_ACTOR и GITHUB_TOKEN, разрешение packages: write задано в workflow; секреты не записываются в репозиторий.

4.7. Ночная проверка

Дополнительный workflow .github/workflows/nightly.yml запускается ежедневно по cron в 01:17 UTC, а также вручную через workflow_dispatch. Он выполняет чистую сборку и модульные тесты, но не создаёт Release и не публикует пакет.

4.8. Проверка и ограничения текущей среды

Исходники приложения скомпилированы локальным JDK 21 и проверены отдельными smoke-тестами. Полный запуск Gradle и JUnit в среде подготовки выполнить не удалось: в ней отсутствовали установленный Gradle и доступ к сети для загрузки дистрибутива и зависимостей. Поэтому окончательную проверку задачи Gradle build и публикаций необходимо подтвердить фактическим запуском GitHub Actions после отправки проекта в удалённый репозиторий.

Примечание: в архив включён bootstrap-скрипт gradlew/gradlew.bat, который загружает закреплённый дистрибутив Gradle с проверкой SHA-256. Это не официальный gradle-wrapper.jar. Если преподаватель требует стандартный Wrapper, следует выполнить gradle wrapper --gradle-version 8.14.6 в среде с установленным Gradle, проверить созданные файлы и закоммитить их.

5. Ссылки на результаты

Удалённый репозиторий пока не опубликован, поэтому реальные ссылки GitHub Actions, Releases и Packages ещё не существуют. После создания репозитория pr2-todo-app в личном аккаунте необходимо заменить OWNER на имя пользователя и проверить:

- Репозиторий: https://github.com/<OWNER>/pr2-todo-app
- GitHub Actions: https://github.com/<OWNER>/pr2-todo-app/actions
- Releases: https://github.com/<OWNER>/pr2-todo-app/releases
- GitHub Packages: https://github.com/<OWNER>?tab=packages

6. Вывод

В ходе работы подготовлен Java-проект Todo CLI, реализована логика управления задачами и дополнительные команды clear, done и search, добавлены тесты JUnit 5. Настроены workflow для автоматической сборки и тестирования, создания GitHub Release, публикации Maven-артефакта в GitHub Packages и ежедневной проверки стабильности. В проекте подготовлена локальная история Git с feature-ветками и merge-коммитами. После публикации репозитория и успешного выполнения Actions можно подтвердить работу CI/CD ссылками на реальные запуски и артефакты.